

RESEARCH ARTICLE

Teaching Software Engineering: A Continuously Evolving Perspective

CLAUDIA RAIBULET^{ID} AND FRANCESCA ARCELLI FONTANA^{ID}

Dipartimento di Informatica, Sistemistica e Comunicazione (DISCo), Università degli Studi di Milano-Bicocca, 20126 Milan, Italy

Corresponding author: Claudia Raibulet (claudia.raibulet@unimib.it)

ABSTRACT Software engineering is in a continuous process of change and evolution led by the dynamicity of our world, and hence, by the aim to adapt to and meet the current and future society's needs. From the educational point of view, one of the main challenges consists in *what* is meaningful and useful to teach in a software engineering course to enable students to have a solid background and to address and lead current and future software development. In this paper, we outline the importance of continuously evolving education (e.g., course material, and teaching methods) to keep up with the changes in software engineering. This enables students to learn the actual and relevant topics, skills, and technologies in a continuously evolving field where new methodologies, technologies, and best practices are constantly being added. We consider this a valuable development in software engineering education, as it allows for an active and relevant learning experience for students. The novelty of our paper consists of 1) a continuous evolution perspective on software engineering education, 2) exploiting various types of inputs driving evolution, and 3) a set of concrete examples and guidelines that can be generalized and applied to similar courses or in other fields.

INDEX TERMS Software engineering, project-based course, continuous evolution, feedback.

I. INTRODUCTION

Teaching software engineering is challenging because it implies a balance among various aspects in a course design [1], [2], [3]. Theory and practice should cohabit. Technical and soft skills should be addressed. Time and resource management should emerge. Students should be prepared to face software development issues in industry and to propose new approaches for upcoming issues. Based on these challenges, we describe our approach to teaching software engineering and its continuous evolution driven by our desire to improve its content and its delivery, in order to meet the students' expectations and industry requirements for software development.

Additionally, there is no unified well-established consensus on *what* should be taught in a course entitled Software Engineering and *how* [4], [5]. There are courses providing an overall view of the discipline, while there are courses focusing more on design and implementation, or testing.

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana^{ID}.

For example, the *First International Workshop on Designing and Running Project-Based Courses in Software Engineering Education (DREE)* co-located with the International Conference on Software Engineering (ICSE) in 2022 aimed to gather various experiences in project-based courses in Software Engineering with the objective to have a common base for teaching such courses.

In our previous works, we focused on stimulating the collaboration and teamwork of students in our Software Engineering course in the third year of the undergraduate program in Computer Science at the University of Milano-Bicocca, in Italy [6], [7], [8], [9]. To achieve this objective, we have introduced (1) the development of a software project in teams as an exam for the software engineering course, (2) the use of supporting tools (e.g., GitHub), and (3) asked for student feedback to evaluate their perception on collaboration and teamwork. In [6] (a one-page extended abstract), we introduced the use of GitHub and the positive student feedback for this tool during the 2016/2017 academic year. In [7], we have described in detail the collaboration and teamwork in the software

engineering course, the use of three tools, i.e., GitHub, SonarQube, and Microsoft Project as support for the exam project, and the students' feedback on their collaboration and teamwork. The results from these previous works, indicate that students are enthusiastic about using various tools for their project development. However, their understanding of the importance of collaboration and teamwork, and the related support provided by the tools, should be improved. Results indicate that some functionalities (e.g., management of task distribution and communication in GitHub) were not understood appropriately, since not all the students used them during the project development. Students' feedback enabled us to gather these issues and consequently, address them more deeply in the following course editions.

In this paper, we focus on the continuous evolution perspective of the Software Engineering¹ course, starting from the academic year 2015/2026. Figure 1 resumes the data we collected over the past years and the different stimuli we received that led to the evolution of the course over time. We consider three different points in time (blue circles), each reporting the contents adopted in the course (white boxes). Starting from the left, we consider the Academic Year (AY) 2015/2016, 2020/2021, and the present one. The course content evolved depending on different factors (blue arrows): the addition of subjects related to DevOps practices was demanded by the increasing interest towards such themes, observed both in academic and industrial settings; the additional proposed tools and the introduction of Design Principles were due to our observation of the feedback we received directly from students (via questionnaires) and indirectly from the grading of the student's exams and our identification of additional content that may improve the course. From the students' examinations, we also gained useful insights about how to improve our teaching methodology. For the 2020/2021 post-pandemic edition, we questioned students also about the online and in-presence learning experience. This helped us revise and update the course content and our teaching to make them as clear and as smooth as possible.

We also obtained external validation of our approach from companies active in the field of software development and engineering about the tools and concepts that we teach in our course. Each year we invite two companies for seminars during the course and we ask them also explicit feedback on the course content.

Currently, we are working on adding further content related to DevOps [10], [11], with a focus on Infrastructure as a Code (IaaS) practices and on cloud platforms (such as Amazon Web Services and Microsoft Azure).

A. RESEARCH QUESTIONS

Figure 1 shows some main aspects concerning our software engineering course over time. It indicates the main content

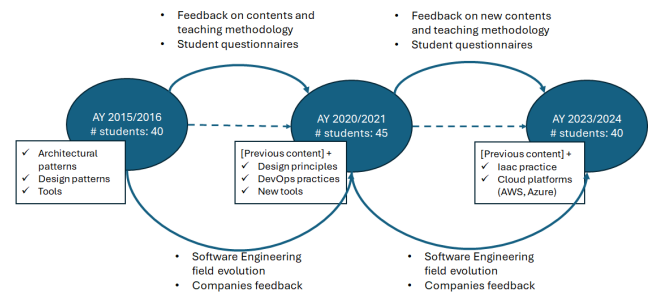


FIGURE 1. SE course evolution - from 2015 to present.

changes (white rectangle), the main sources of feedback on the content evolution, the academic years, and the number of students attending the course. Considering the evolution described in Figure 1, we formulate the main Research Question (RQ) of this paper, further subdivided into two Research Questions (RQ1 and RQ2):

- **RQ: Which are the main challenges faced in a continuously evolving software engineering course?**

We aim to answer this question from the educational perspective by exploiting the data we collected during the past academic years. Our aim is to provide a retrospective study to enhance our teaching activity and plan the next steps. We expand the subject by proposing the following research questions:

- **RQ1: Which is the main feedback on the course through the questionnaires and exams projects?** With this question, we aim to verify how students managed to pass from theory (our lessons) to practice (the development of a solution) and whether they found specific difficulties. In this way, we can understand how to enhance our teaching. To reach this goal, we analyze the feedback of the students about the software engineering practices and tools studied and experimented with during the course. We exploit the data coming from their exam revision and the answers to questionnaires.
- **RQ2: How did the course content evolve during the last decade?** With the answer to this question we aim to outline the main changes we applied to our course, in terms of content and methodology, and to analyze also the different feedback, according to teaching software engineering, which led us to upgrade the course content during the past ten years.

B. RESEARCH APPROACH

To answer these RQ, we have analyzed the course content and its evolution from 2015 until present. Our data consists in the course content, student results, student feedback, and companies feedback. Further, we consider the evolution of the field during these years (e.g., design principles, DevOps, cloud platforms). Each academic year, we analyzed the student results, asked for feedback from the students on the course content, and interacted with the companies providing

¹Software Engineering Course - <https://elearning.unimib.it/course/info.php?id=49468#en>

seminars during the course about the course content. Further, we have compared the results related to the current academic year with the results of the previous ones. Based on the results, we decided what to improve in the content and teaching. This comparison is qualitative and done by the professors of the course.

C. RESEARCH CONTRIBUTION

The contribution of our work can be outlined as follows:

- provide a new perspective on software engineering education by considering the continuous evolution of the course content over time;
- consider various drivers for this evolution, i.e., our desire to improve teaching both from the content and the delivery point of view;
- propose a set of guidelines for the continuous evolution of software engineering courses;
- present a concrete example of the application of these guidelines in a time frame of a decade.

The worldwide interest and the importance of the topic addressed in this paper for software engineering education are confirmed by recent related research for example in teaching computer programming and computational thinking, e.g., from primary schools [12] to undergraduate courses that aim to deliver personalized learning material through advanced approaches which combine instructional theory and intelligent techniques [13]. Further, a review on the worldwide teaching of programming and computational thinking [14] outlines the differences in computer science education on five continents, and, hence, the need for common practices.

Following is the outline of the paper. Section II reports some related works in the field of teaching software engineering. This section is organized into three subsections based on the main topics addressed in this paper: project-based course as a teaching method, use of available tools for software development, and interaction between academia and industry in the context of software engineering courses. Section III introduces the aim, organization, and contents of our software engineering course. Section IV reports and discusses the results of our study. Finally, Section V concludes by answering our research questions and by proposing some future developments.

II. RELATED WORK

In this paper, we consider related work on software engineering courses, both according to methods adopted to teach software engineering and their evolution over time, as well as to the tools taught in these courses. We also consider the interaction between education and industry in the context of software engineering courses.

A. METHODS TO TEACH SOFTWARE ENGINEERING

Teaching software engineering is an open and evolving issue at all education levels as emerges from recent studies [4], [12]. Undergraduate and graduate study programs offer

a wide range of courses on requirements engineering, software engineering processes, software design, software testing, software maintenance, and evolution, in addition to programming ones.

A recent systematic review of the available teaching methods in SE is described in [15]. This research considers the results published in the main journals focused on computer science education (e.g., IEEE Transactions on Education, ACM Transactions on Computing Education) and on the main scientific databases (e.g., Web of Science, IEEE Explore, ACM Digital Library). From an initial set of 2631 papers, the authors have identified 446 primary studies. The review extracted from these primary studies 17 different teaching methods and approaches in SE. The most spread and recognized ones are: project-based learning, agile learning, collaborative learning, active learning, and problem-based learning. The main characteristics of these 17 methods are teamwork, teacher as a facilitator, simulation of a working environment, and collaboration with industry. Our teaching method is perfectly aligned with the survey results.

B. TOOLS IN SOFTWARE ENGINEERING COURSES

Tools are adopted for various reasons in software engineering courses, e.g., development support, distributed development support, development history trace (versioning software), teamwork environment, software quality assessment, and project management. Due to this fact, that there are various tools that are used in the context of courses, in this section we focus on those tools we have considered in our software engineering course: GitHub, SonarQube, Understand, Travis, and Microsoft Project.

1) GITHUB

Feliciano et al. [16] and Zagalsky et al. [17] use GitHub for hosting course content and student assignments in software engineering courses at the University of Victoria, Canada. They consider both the instructor's and the student's perspectives. Instructors and students had no previous experience in using GitHub. The benefits of using GitHub in software engineering courses outlined in [16] resulted also from our experience. Additionally, we used GitHub for a software development project for an exam and the instructors had already used GitHub before in software engineering research activities [18]. Francese et al. [19] and Francese et al. [20] present a similar work to ours in the context of a Mobile Application Development course. GitHub has been used as a collaborative development platform by the students to develop their projects counting also as the exam for their course. The authors required the student feedback on the developed project through 9 questions. Francese et al. focused on the development of a mobile application while we investigated the effectiveness from various points of view of using GitHub and other tools in the context of a software engineering course.

Patani et al. [21] makes an investigation on the impact of using GitHub as a tool in a software engineering course on

students' development skills. Particularly, the paper considers aspects such as course performance, interest in using it, and students' confidence, comfort, and anxiety levels related to it.

2) SONARQUBE

Bai et al. [22] indicate that SonarQube is used by professors to analyze the code written and submitted by students for assignments in a programming course. A similar work is presented in [23] which describes the improvement of the student code after introducing two additional aspects in a programming course: portfolio assessment and test coverage. The code was analyzed with SonarQube by the professors. Falessi and Kruchten [24] outline the importance of introducing the notion of technical debt in the software engineering curriculum. As an example of tool that they propose to the students to assess the quality of their code is SonarQube. Delgado et al. [25] presents the evolution of a project-based software engineering course during 3 years. The project-based course was focused on the application of agile practices during the development of a project in teams. The development framework for the projects changed in time from Grails to Rails. When Grails was used, also SonarQube was used by the students to analyze their software quality. When Rails replaced Grails, SonarQube was abandoned. Students were asked to provide their feedback on the application of the agile process. de Andrade Gomes et al. [26] introduces the SMaRT tool for teaching software quality aimed to help students improve their code and programming skills. SMaRT uses SonarQube and Eclipse IDE. Rong et al. [27] introduce DevOpsEnvy, an education support system for professors. Through DevOpsEnvy, professors manage and monitor the student teams practicing DevOps. This system integrates various open source tools such as GitHub, Jenkins, SonarQube, Maven, Gradle, and Docker. Essentially, in de Andrade Gomes et al. [26], Bai et al. [22], Lauvas [23] and Rong et al. [27] SonarQube is used by the professors to assess the quality of the individual work of the students in the context of the programming courses. Falessi and Kruchten [24] use SonarQube to analyze the technical debt of their individual work. Delgado et al. [25] mention the use of SonarQube for software analysis in an agile project-based course and do not investigate the advantages or limitations of using it during the project development.

3) MICROSOFT PROJECT

In Groth and Hottell [28] students plan tasks for a capstone project course by using Microsoft Project tool. Groth and Hottell focus on the definition of a capstone project course and its evolution in time, rather than introducing project management activities in the context of a software engineering course. Car et al. [29] describe the introduction of a project management course for undergraduate and graduate students. Similarly, Stoica and Islam [30] introduce a project management course for undergraduate students together with a basic and an advanced course on software engineering.

Garcia et al. [31] presents an experience of teaching a project management course for undergraduate students. The objectives of these courses are significantly larger than the introduction of the basic notions of software project management in our software engineering course. However, we outline that also these courses use Microsoft Project tool.

We did not find relevant research on using Travis and Understand tools in the context of teaching software engineering, tools that we experimented with and discussed in this study.

C. INTERACTION BETWEEN EDUCATION AND INDUSTRY IN SOFTWARE ENGINEERING COURSES

The interaction between education and industry is also a key aspect considered in software engineering courses. This is extremely valuable for students because they may experience a flavor of work aspects, and for education because it helps address current market needs. *What* and *How* to teach software engineering are subject of continuous research.

Cico et al [32] investigate the extent SE education addresses the major SE trends in project based courses. Their findings indicate that trends are introduced and assimilated through project based learning.

Schneider et al. [33] propose a novel approach for the organization of the final exam project in software engineering and report the preliminary outcome of its adoption. The approach consists of adopting an agile organizational model to produce software artifacts with incrementally changing team members working on a product-line, to simulate a professional work scenario. Students are flanked by professional software engineers and also the professors have responsible roles inside the projects (e.g., product-line manager). The model aims also to make students develop soft skills, such as leadership and teamwork. Still related to the teaching of software engineering, but from a different point of view, Ryan [34] describes his experience in teaching software engineering students *professional ethics*. In particular, for 20 years he collected the answers to an "Introductory Quiz" consisting of 10 short questions representing a potentially problematic (in terms of ethics) work assignment. Additionally, to present the results of his study, he proposes a set of topics that in his opinion should be taught to students to complement their education.

Other studies investigate the relationship between academic education and industry needs, i.e., what students are expected to know when entering the job market. For instance, Radermacher et al. [35] conducted an empirical study on 23 managers and human resources personnel from various software companies active in United States and Europe to investigate the gap between student education and industry expectations. Results show that students particularly struggle in using software tools, such as configuration management systems, and in programming unit tests. Moreover, students lack communication skills, both towards their colleagues and their clients, as much as they lack problem-solving abilities.

Similarly, Vanhanen et al. [36] report the results of a survey conducted on the students of their educational projects, which in their case involve real industrial customers. Given that students may encounter problems of various nature during the project development, the authors study the relationship between such problems and customer satisfaction. Results underline that most of the problems are related to testing, task management, effort estimation, and technology skills. They also acknowledged that if students are supported by their supervisors (professional developers and managers) in solving their problems, they learn more than their unsupported peers.

Finally, Ciro et al. [32] conducted a systematic mapping study about teaching major software engineering trends to investigate the gaps between industry and education. From their review, they found out that agile software development is the major trend in academic courses.

With respect to the outlined related works, we do not propose new teaching methods. Our aim to underline the need for the continuous evolution of a software engineering course is led by two main reasons: (1) to improve the students' learning experience in software engineering based on their direct feedback and on our perceptions during the exams, and (2) to prepare students for the software development after their graduation by considering the needs of the industry and the advances in software engineering.

To make a comparison with the software itself (see Lehman's software evolution laws [37]), if a software engineering course does not continuously evolve (e.g., change and grow) and adapt, it becomes progressively less useful and efficient over time. Therefore, in this paper, we introduce our course topics and the motivation behind them, collect internal feedback data of the past years coming directly from the students, consider the feedback received from the companies, and analyze how the improvements added along the course evolution have impacted the students.

III. OUR SOFTWARE ENGINEERING COURSE

This section outlines the main characteristics of our Software Engineering (SE) course, comprising the general aim, the course contents, and the exam organization. Concerning the course contents, the description splits into two: Section III-B addresses contents taught starting from 2015/2016, while Section III-C introduces the new subjects and the new tools taught starting from 2020/2021.

A. THE AIM OF OUR COURSE

The SE course at our department aims to teach advanced knowledge, principles, techniques, and tools for software development and assessment. The main contents of the course concern architectural patterns, design patterns, design principles, best practices in the Java programming language, tools and frameworks for software development, as well as software quality assessment through software metrics, code and architectural smells using tool support. A particular

attention is provided to the introduction to DevOps and Continuous Integration.

1) HOW IS THE SOFTWARE ENGINEERING COURSE COLLOCATED IN THE CONTEXT OF THE UNDERGRADUATE PROGRAM?

In our university, the basic software engineering concepts are introduced in the two *Java Programming* courses during the first year, one in the first and one in the second semester. During the second year, the *Software Analysis and Design* course is mandatory for all students in the second semester. Various development methodologies are introduced (e.g., Waterfall, Spiral, SCRUM, Unified Process (UP), Rational Unified Process (RUP) based on the UML concepts). Students learn about patterns, i.e., GRASP (General Responsibility Assignment Software Patterns) [38] (e.g., Creator, Controller, Low Coupling, High Cohesion, Information Expert) and Gamma's et. al. [39] Design Patterns (e.g., Singleton, Adapter, Facade, Composite, Observer, Strategy).

In the first semester of the third year, the optional *Software Engineering* course, central to the present research, is collocated. This course aims to introduce advanced software development concepts often used in software projects, and to simulate a working environment for the development of a software project in teams.

2) HOW DO WE ORGANIZE THE SOFTWARE ENGINEERING COURSE?

The course is a semester-long. It is composed of 2 hours of lessons, 2 hours of exercise sessions, and 2 hours of labs per week for a total of 12 weeks. The course is continuously enhanced with new topics or re-organized based on the students' feedback and advances in SE. Some major changes have been made during the 2016/2017 academic year by introducing mechanisms, activities, and tools to encourage collaborative and teamwork software development.

During the exercise sessions, we propose the development of software requiring the application of patterns (see Section III-B) and design principles (see Section III-C). We outline the advantages of the use and reuse of patterns. We present available implementations (e.g., a persistence framework) based on patterns and Object-Oriented principles. We also introduce challenging types of examples such as self-adaptive systems underlying their design issues. Additionally, students are asked to propose solutions for management aspects of software projects. Afterwards, project management basic concepts are presented and a couple of related examples are discussed.

Concerning the lab sessions, we offer an overview of the tools and practices at the basis of the software development life-cycle (see Section III-C2 and Section III-C3) and we make students practice them during the lab sessions and exam.

B. COURSE CONTENTS

1) ARCHITECTURAL PATTERNS

We introduce the principal aims of architectural patterns and in particular, the patterns often used in enterprise applications, by following Fowler's book [40]. Then we show the students a list of architectural patterns in different categories: Domain Logic, Data Source Architectural Patterns, Object-Relational Behavioural and Object-Relational Structural Patterns, Web Presentation, Concurrency Patterns, Security and Performance Patterns. We ask each student to select one pattern to study in detail in order to present the pattern using slides during a lesson. Based on the lesson hours dedicated to this topic and on the number of students attending the course, we usually schedule two lessons of two or three hours each, where each student is invited to present the architectural pattern in 7-8 minutes, by outlining the principal aim of the pattern, one example of application of the pattern and pro-cons of the pattern. Giving exact and short time for each presentation, students can learn how they can exploit the given time by focusing only on the most relevant features of a pattern. Students provide the slides presentation of each pattern and other possible contents related to the patterns to be uploaded to the Moodle website - the course management system at our university.² Obviously, all the materials are checked by one of the professors of the course. In this way, each student can consult this material if needed. In case not all the students can present the pattern for time reasons (we have about 40-45 students attending the course and not enough time for all the presentations in two lessons), we ask them to provide in any case the slides and we check them during the oral examination.

Aims for the exam project: During the exam project, students are invited to exploit, if necessary, one or more of the architectural patterns according to the aim of the exam project. They outline in the document attached to each project the architectural patterns they used, if any, and the reason for the adoption of the pattern(s) in the project.

2) DESIGN PATTERNS

In the SE course, during exercise sessions, we stress the reusability of design patterns, their semantics, and the role played by the semantics in the developers' vocabulary and project documentation. Moreover, we present other seven patterns, i.e., Abstract Factory, Factory Method, Build, State, Command, Template, and Chain of Responsibility in addition to those already presented during the second year.

To outline the advantages of the design patterns, we propose examples of problems that may be solved/implemented with and without the application of patterns. The examples address classical problems such as the design of an alarm clock requiring the State pattern, of a remote control requiring the Command pattern, or the building of various models of cars through the Abstract Factory. We ask students to

provide their solutions to these problems in terms of UML diagrams and code. Very few students (e.g., just one or two per year) unconsciously provide an original solution which ends up to be the exact design pattern. In addition, we observe that students are always tempted to write directly code and skip the design phase or use UML diagrams to model their solution even if they are in the context of a SE course. Some of the design pattern examples are taken from Craig Larman's book on *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* [38].

Aim for the exam project: The exam projects are specified in such a way that at least a couple of design patterns may be easily applied. Students have to indicate the use of patterns and the reason for their use, as well as document their application in the code.

3) TOOLS

The tools we introduced since the 2016/2017 edition of the course are: GitHub, SonarQube, and Microsoft Project. Students learn and use these tools during the labs and exercise sessions.

- *GitHub*³ is adopted as a collaborative development platform for software projects. It has been presented to the students during the first laboratories, along with the introduction to Git, and further used during the entire course. The following functionalities are presented: branches, staging, commit, push and pull functionality, and conflict resolution.
- *SonarQube*⁴ is adopted as a software quality assessment tool for Java projects. It has been presented to the students during the laboratories. The focus is on the installation, configuration, and use of SonarQube (e.g., the various plug-ins of SonarQube, the configuration of the database used by SonarQube, the analysis of Java projects through the Maven plug-in). Students also performed the analysis of various available Java open source projects during the laboratories.
- *Microsoft Project*⁵ is adopted as a tool for software project management. During two seminars, students are taught the basic concepts of project management, focusing mostly on project life cycle and organization, time, cost, human, communications, and risk management (as described in [41]). Furthermore, the students are shown two simple case studies and asked to do themselves a case study using Microsoft Project.

Aim for the exam project: *GitHub*: Each team receives a GitHub repository to manage the source code of the project. Each team member receives access instructions to his/her repository. Students are notified that they will be evaluated based on: the way they use GitHub for the code management, the use of branches, and the types of uploaded files. Each

²Moodle <https://moodle.org/> is the Course Management System used at the University of Milano-Bicocca <https://elearning.unimib.it/>

³GitHub - <https://github.com/>

⁴SonarQube - <https://www.sonarqube.org/>

⁵Microsoft Project - <https://www.microsoft.com/>

team member is evaluated based on the way he or she uses the repository, i.e., not only considering the number of operations performed, but also their quality.

SonarQube: We ask students to use SonarQube to evaluate and monitor the software quality of their projects. At the end of the project, before its delivery, they have to indicate the setting/configuration necessary to analyze their project with SonarQube and include the configuration files in the project (e.g., pom.xml and sonar-project.properties). Moreover, we ask students to deliver the project with a rating level of A (the maximum) among the different indicators available in SonarQube and without critical violations, always according to the violation classification provided by SonarQube. We let students choose which SonarQube version to use, either the on-premise version or the cloud version (SonarCloud, see Section III-C2). In particular, the latter can be combined with the usage of Travis (Section III-C2), to create a continuous integration pipeline.

Microsoft Project: We ask students to do a Gantt diagram at the beginning of the project development with the estimation of the effort required. This initial diagram has to be sent by email to the professors during the first week of the project development. Further, students have to update the Gantt diagram during the entire project and send to the professors the final version once they finished the project. We ask for these two versions to see how close their initial estimation is to the real effort, and to make them aware of the gap between planning and developing the project.

C. EVOLUTION OF THE COURSE: NEW CONTENTS

1) DESIGN PRINCIPLES

We introduce different design principles and best practices, in particular, the ones of Martin [42], focusing our attention on the principles of Object-Oriented class design and package architecture. Then, we outline the SOLID principles of Object-Oriented programming [43]: Single Responsibility Principle, Open Closure Principle, Liskov Substitution Principle, Interface Segregation Principle, Dependency Inversion Principle. Finally, we introduce PHAME: Principles of Hierarchy, Abstraction, Modularization, and Encapsulation proposed by Suryanarayana et al. [44]. The authors outline that the violations of these principles could lead to the introduction of different smells, at code, design, and architecture levels. In the course, we focus our attention in particular on the problems that could arise through the violation of the Modularization principles, such as all the problems related to dependency issues. In this direction we present the structural anti patterns (SAP), sometimes also called architectural smells (AS), based on dependency issues at class and package level: HubLike, Breakable, Butterfly, and Tangle (also called Cyclic Dependency) antipatterns. According to the structural antipatterns, we introduced a tool, called Understand (described in Section III-C2), able to detect some of these problems.

Aims for the exam project: We ask students for the exam projects to outline if they followed some of the design principles, or if they think that they violated a particular principle, as a kind of self-admitted technical debt [45]. Moreover, they have to analyze their project through the Understand tool and check if some antipatterns have been detected by the tool (see Section III-C2). In this case, we ask students to try to remove through refactoring steps the encountered problems (i.e., antipatterns).

2) NEW TOOLS

In this section, we describe the tools introduced since 2020/2021 academic year: SonarCloud, Travis, and Understand.

- **SonarCloud**⁶ is the cloud version of SonarQube. It is natively integrated with the most famous Git repositories providers (GitHub, Gitlab and Bitbucket) and offers integration also with Continuous Integration pipelines, such as the ones of Travis and Azure.
- **Travis**⁷: is adopted as hosted, distributed continuous integration service used to build and test software projects hosted at GitHub.
- **Understand**⁸: is adopted as a software analysis tool to assess various quality metrics and to graphically visualize the structure of a project and detect some structural antipatterns.

Aim for the exam project: SonarCloud: The use of SonarCloud in terms of code analysis is equal to the one of SonarQube, described in Section III-B3. Indeed, we let the students choose which tool to use. Usually, students choosing also to adopt Travis, prefer to accompany it with SonarCloud, to enable automatic code analysis in their CI pipelines.

Travis: Travis is part of the teaching of DevOps practices (see Section III-C3). Due to the difficulties students usually encounter while setting it up and given the limited time we give them to design, implement and test their projects, we prefer to let students decide whether to adopt Travis or not. However, in the case students use it, we check the Travis configuration file and whether they correctly managed the build pipelines. For instance, we check if they indiscriminately run the project pipeline each time a new piece of code is committed to GitHub or if they schedule the runs depending on some logic (merge of branch on master, milestone, and so on).

Understand: We ask students to analyse their project with Understand and to check for the presence of antipatterns (see Section III-C1) and, if present, to remove them. Students must attach the tool report in their project documentation.

3) DEVOPS PRACTICES

The SE field, as the entire computer science environment, is in constant transformation: teaching must move forward

⁶SonarCloud - <https://sonarcloud.io/>

⁷Travis - <https://travis-ci.org/>

⁸Understand - <https://www.scitools.com/>

to offer updated content and inspiring ideas. A topic that emerged during the past few years, but which is still not part of our students' background, is the *DevOps* culture [46], [47].

We used to present GitHub and SonarQube separately and teach how to use them in isolation. However, since 2020/2021 we introduced our students to *Continuous Integration* (CI) practices [48] and in particular the CI pipelines [49]. We showed students how different tools, GitHub and SonarQube, can empower and integrate each other inside a pipeline, and why this practice is becoming popular in both research and industrial settings.

The new concepts and technologies we introduced are:

- *DevOps*: DEvelopment and OPerationS, is a software development methodology whose aim is to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives.
- *Continuous Integration (CI)*: is the practice of merging in small code changes frequently rather than merging in a large change at the end of a development cycle. The goal is to build healthier software by developing and testing in smaller increments.
- *CI pipelines*: is the sequence of steps to be followed and tools to be used to automate the integration and testing of code changes.
- *Travis*: CI platform, see Section III-C2.
- *SonarCloud*: is the cloud service offered by SonarSource and based on SonarQube, see Section III-C2.

The lessons during the lab sessions are balanced between theory and practice, where students have the opportunity to experiment new technologies. We evaluate students on the use of the new introduced tools with dedicated exercises, where they have to build CI pipelines. Moreover, we encourage students to adopt such tools during the exam project development.

Our teaching schedule concerning DevOps is the following:

- Introduction to the DevOps culture. In order to provide context to our lessons and explain what pipelines are, we set up a case study story: *"Imagine being part of a team running an innovative start-up for the development of an app for searching social and cultural events in your town. Your prototype is already on the market and you are receiving positive feedback. However, your clients grow every day and so do their needs: now they want to connect with other users from inside your app and filter the search results by price. Hence, you have to coordinate with your team, manage new multiple features at the same time, and deliver a functional, reliable product. How do you keep up with the constant requests without harming the quality of your codebase?"*
- Introduction to the technologies. The focus is on GitHub, SonarQube, SonarCloud and Travis, with quick refer-

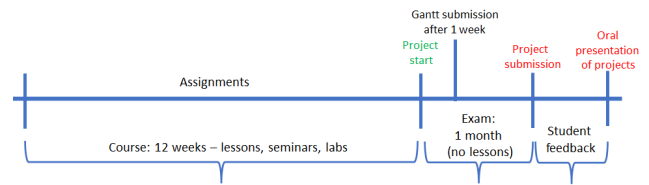


FIGURE 2. Software engineering course: Assignments and exam timeline.

ences to available alternative solutions such as Gitlab⁹ and Bitbucket¹⁰ for version control and Jenkins¹¹ and Buddy¹² for CI.

- Technical example with Travis, GitHub, and SonarCloud. We introduce an example of the Travis CI pipeline which connects to GitHub and when triggered by the push of a commit to a remote repository, automatically 1) gets the last version of the project from Github, 2) and runs SonarCloud analysis.
- Exercises focused on learning how to configure the different tools and on how to integrate them.

Aim for the exam project: We push students to adopt the introduced DevOps practices during the development of the exam project, however, we do not require the use of the CI tools: currently the focus of the project is on the design and development of complete software applications, not on DevOps practices. In the future, we plan to lighten the requests concerning the project implementation and give more room to the experimentation of DevOps practices.

D. EXAM ORGANIZATION AND EXAMPLES OF EXAM PROJECTS

Students are asked (1) to complete several assignments during the labs counting for the final evaluation and (2) to develop a software project in teams by applying the mechanisms, knowledge, principles, and tools seen during the course. Figure 2 depicts the timeline concerning assignments and exams.

Assignments: Starting from the 2020/2021 academic year, four assignments have been proposed, i.e., create Git projects and use Git functionality, build a pipeline with Travis, create Maven projects, and remove quality issues from projects. Assignments are individual. They should be fulfilled in one week and before the end of the teaching period. Through assignments, we aim to motivate students to actively follow the course topics and arrive prepared at the end of the course for the project development. From assignments, we understand some of the difficulties students may have in assimilating the learning and we are in time to discuss further unclear aspects.

Exam: Students are asked to organize themselves in teams of 3 to 5 students. In this way, students are encouraged to

⁹Gitlab - <https://gitlab.com/>

¹⁰Bitbucket - <https://bitbucket.org/product/>

¹¹Jenkins - <https://jenkins.io/>

¹²Buddy - <https://buddy.works/>

take action and be responsible and choose their team members based on their preferences and the academic knowledge/skills of their mates. The number of students in teams depends on the number of students attending the course and on the potential workload of the professors in grading the projects. In our opinion, the ideal number is 3 (or max 4). An odd number enables the project related decision. We delegate the teams' organization to the students because of several reasons. First, students should become aware of their responsibility. Second, they cannot complain about their teams. Third, based on our experience, if we organize teams then a lot of issues should be managed by professors (e.g., unequal contribution to the project, students quitting teams).

Students are required to use 5 tools for the management and development of the project: GitHub, SonarQube, Understand, Travis, and Microsoft Project (see Section III-B3 and Section III-C2 for more details). These tools are introduced during the course and we expect students to be able to use them easily.

Each team receives a different software project to be developed, however, projects are similar in requirements and workload. We have decided on different projects to avoid exchanges of solutions among different teams. The workload for each team and each team member is comparable.

Students have 1 month time to develop (i.e., analyze, design, and implement) the software project from the moment they receive the project description. Students are required to provide documentation about the project. The documentation should include the UML diagrams (i.e., use case diagram, the domain model as a class diagram, the package diagram, the class diagram of the solution, activity diagrams, interaction and state diagrams). UML diagrams have to be designed with a tool such as Rational Software Architect (RSA). Furthermore, in the documentation, they have to outline if they applied architectural or design patterns. Whenever the project description is not clear or complete, students are allowed to make assumptions described in the final documentation. Within a week from the beginning of the project, students are asked to send by email a Gantt diagram with an estimation of the project activities and duration. The Gantt has to be updated during the project development according to the real effort needed by the project. The final version of the project is required to be executable and testable by the software engineering professors/tutors. This is mandatory for the evaluation of the project. Furthermore, students submit documentation about the installation and execution of their project. To finalize the delivery of the project, one member of each team has to send an email, "Delivery-email", with the GitHub *commit tag* to be considered for the delivery and a pdf file with the UML and installation documentation, as well as the final version of the Gantt diagram. At the end of the project, students receive by email links to five questionnaires on GitHub,¹³ SonarQube,¹⁴

Understand,¹⁵ Travis,¹⁶ and Microsoft Project¹⁷ to be filled in.

Finally, after the evaluation (usually one week from the delivery of the project by the students), each team sustains an oral exam session with the discussion on their project with questions or comments on how they used the above-mentioned tools, on the documentation of the project (the UML diagrams), on the design choices (architectural patterns or design patterns adopted) and on any other issue/problem related to the project development or execution. We highlight that this software project is (1) the first project the students develop completely from the analysis to the implementation and evaluation for an exam and (2) the first team project in our BsC curriculum where they collaborate to reach a common goal: to pass an exam. We also would like to highlight that the final evaluations are individual, reflecting the contribution of each student to the development of the project that emerged during the oral session, on how they used GitHub, and on the individual evaluations of the exercises assigned during the lab sessions. As mentioned also in Delgado et al. [25] and Rajlich [50], it is very important to adopt mechanisms enabling individual grades in team projects in order to maintain fairness.

We briefly describe below examples of projects proposed for the exam since the 2020/2021 academic year, selected from the 43rd International Conference on Software Engineering (ICSE2021)¹⁸ for the SCORE¹⁹ contest. SCORE contests are held by ICSE every 4 years and gather projects defined by SE practitioners and researchers. For example, among the projects, the first one is related to the pandemic context.

*CLup: CustomersLine-up:*²⁰ The Covid19 emergency has put a strain on society on many levels, imposing lockdowns and enforcing rules even when people are justified in going out (e.g., limiting the number of accesses to buildings). In particular, grocery shopping—a most essential need—can become a challenge in the presence of such rules. Indeed, supermarkets need to restrict access to their stores to avoid having crowds inside, which results in long lines forming outside, which are themselves a source of hazards. The goal of this project is to develop an easy-to-use application that allows stores to regulate the influx of people in the building and saves people from having to line up and stand outside of stores for hours.

*PladDat: Placement Dating for Students:*²¹ This project creates a matching service to connect placement students with potential employers. Using a "dating" service metaphor,

¹⁵Link to Understand Questionnaire

¹⁶Link to Travis Questionnaire

¹⁷Link to Microsoft Project Questionnaire

¹⁸<https://conf.researchr.org/home/icse-2021>

¹⁹<https://conf.researchr.org/home/icse-2021/score-2021>

²⁰<https://conf.researchr.org/home/icse-2021/score-2021#clup-customers-line-up>

²¹<https://conf.researchr.org/home/icse-2021/score-2021#pladat-placement-dating-for-students>

¹³Link to GitHub Questionnaire

¹⁴Link to SonarQube Questionnaire

the project allows employers to post details of placements and students to apply for placements. This project creates a database-driven mobile-friendly website to encourage employers and students to engage in year-long paid placements as part of an honours degree programme.

Moreover, we assigned also other projects in addition to the SCORE projects as the ones described below.

Smart Energy for Your Home: More and more home appliances are becoming smart devices, which can be monitored and controlled remotely. For example, they can receive commands for the scheduling of their working cycles, or to safely interrupt them (e.g., a washing machine allows its washing cycle to be interrupted for a limited time without compromising the quality of the washing and without damaging the clothes); these commands can be used to avoid system overloading, which can in turn lead to blackouts, or to increased costs when certain thresholds are crossed.

Smart Home: This project should support residents by automatically adjusting certain parameters of the house according to various measurements and rules based on the residents' behavior and habits. Therefore, the system monitors various parameters and actions (such as temperature, light intensity, movements in the house, gas use, cleaning related activities, or security/alarm prevention) and uses actuators to influence such values (e.g., turn on the heater, close the shutter, or perform an emergency call).

Finally, other recurring projects we propose every year consist of the development of Web applications similar to those used by students in their everyday life, such as:

- booking and selling of cinema/theatre tickets;
- booking and selling of train/flight tickets;
- booking of cars or bikes in a car/bike sharing system;
- booking of rooms in a hotel;
- management of services offered by a gym/fitness center;
- management of the papers submissions for a conference/journal;
- implementation of games such as Risiko and Monopoly.

IV. FEEDBACK ON A COURSE IN A CONTINUOUS EVOLVING PERSPECTIVE

In this section, we summarize and discuss the various types of feedback we gathered on our SE course to answer our RQs. Feedback is fundamental to continuously evolving and improving courses. In our case, feedback comes mainly from (1) the exam projects and the subsequent oral sessions which summarize the knowledge and experience acquired by the students during the course, (2) from the questionnaires we ask students to fill in after the exam projects have been submitted and before the oral presentations, and (3) the IT companies invited during the course to provide seminars on SE topics. Hence, we gather indirect feedback through projects, direct feedback through questionnaires, and third-party feedback through IT companies.

The number of students attending the Software Engineering course each year is around 40/45. Hence in the

last decade, we have received feedback from around 400+ students. Further, we invite one or two companies for the seminars each year. Hence, we have received feedback from 10+ IT companies.

Each of the following sections addresses a specific research question, as indicated in the title of the section itself. The final answers to the questions are reported in the conclusions.

A. RQ1: FEEDBACK THROUGH THE EXAM PROJECTS

The exam projects provide implicit/indirect feedback of the students on the course content, on the understanding of the course content, and on the application/use of the course content in practice (the translation/application of the theory into practice). They are key sources in identifying both positive aspects (e.g., students apply the taught concepts) and the current limitations/challenges (e.g., difficulties in using tools or applying taught concepts). We consider it an implicit feedback because through the projects we are able to capture which are the consolidated topics/concepts, which are the still developing or emerging topics/concepts, as well as what concepts may be further introduced/removed for the exam projects to improve their quality and achieve their learning objectives.

We let students choose how to design their solutions and most of the teams decided to develop a web application (exceptions are a command line tool and a desktop solution written in .NET). Table 1 reports the tools adopted by students for the development of the exam projects. We indicate the number of adoptions next to the name of the technology. The most chosen programming language for back-end development is Java, whereas the most chosen one for the front-end development is Javascript. MySQL is the most adopted database; Hibernate and Springboot are the most adopted (Java) frameworks. The reasons behind their choices are the same for every team: they already knew them because of past experiences during both university courses and work experiences.

With respect to the aims of the exam project and what we asked students to outline (described in Section III), we observed that students used different design patterns according to the different aims and scope of their projects. In particular, they applied the following design patterns: Observer, Singleton, Facade, Strategy, and in few case cases also Builder.

According to architectural patterns, they applied: MVC, Domain Model, Data Mapper, Active Record, Data Access Object, Interceptor, Identity Field, Association Table Mapping, and Service Layer.

The design principles often applied by the students are the following: Lisk Substitution Principle (LSP), Acyclic Dependency Principle (ADP), Interface Segregation Principle (ISP), and Single Responsibility Principles (SRP). A few of them applied also the Dependency Inversion Principle (DIP) and the Open Closed Principle (OCP).

For the structural antipatterns, students considered Understand useful for their detection, even if few students tried

TABLE 1. Adopted approaches and technologies.

Language		Libraries/Frameworks		Platform	Database
Backend	Frontend	Backend	Frontend		
Android (1), Java (7), .NET (1), Javascript (1)	JavaScript (3), TypeScript (1), Java SWT (1)	Hibernate (2), Install4j (2), Lombok (1), NodeJS (1), SpringBoot (3), Tomcat (1)	Angular (1), JSP (2), React (1), visjs (1), vuetify (1)	AWS (1), Confluence (1), Docker (1)	SQLite (1), MySQL (5)

to remove them through the refactoring of their project. Moreover, they found Understand useful for the computation of some software quality metrics and particularly appreciated the Understand Tree Map view, which they exploited mostly to identify large and complex classes. However, we observed that students usually are confused about capturing the usefulness of some quality metrics, hence in the future we plan to better outline the importance and the differences among the metrics computed by Understand, and invite students to focus only on the most relevant ones.

Finally, another feedback we want to outline according to the previous student knowledge on UML,²² is that unfortunately few of them correctly design software using UML diagrams. Most of them make many mistakes by mixing state and activity diagrams, and also mistakes in the domain model diagram and in defining the logical architecture diagram. Hence, we have to better emphasize these contents in the bachelor course at the second year.

B. RQ1: FEEDBACK THROUGH QUESTIONNAIRES

We have asked students for explicit/direct feedback on the course through questionnaires since several years. We have started with feedback on the use of GitHub during labs and exam project development. We enriched the questionnaire on GitHub with new questions and asked for feedback also on the use of SonarQube and Microsoft Project in the following years.

Since the 2020/2021 academic year, we have extended the direct feedback with two more questionnaires concerning the use, role, and advantages of Understand and Travis during labs and exam projects. In the following, we summarize the feedback gathered through the five questionnaires. For the three of them described in detail in citeJSS2018, we indicate the trend in time, while for the other two, we provide details in this section. In the appendix of this paper, we also included figures that summarize the differences between the feedback received in 2018/2019 and 2020/2021 for some of the questions concerning GitHub, SonarQube, and Microsoft Project, in order to outline some trends.

1) GITHUB

The feedback on GitHub asks for general information (i.e., awareness of its existence and its eventual previous use), technical skills (i.e., knowledge on GitHub functionality), communication and teamwork support (i.e., communication support among the team members, tasks distribution). Currently, the GitHub questionnaire counts 37 questions (12 more than in 2018/2019) being in a continuous evolution as the entire course. The awareness concerning the existence of GitHub is growing: from 73.3% in 2018/2019 to 81.8% in 2020/2021. This growth is not reflected also in the use of GitHub before the SE course: from 50% in 2018/2019 to 25.9% in 2020/2021. We note stability in the familiarity level with GitHub before and after the development of the project exam and in the usefulness of GitHub during the project development. Students already familiar with GitHub before this course mentioned that they used GitHub during high school projects, work experiences, or own projects developed individually or together with mates. However, we observe that the familiarity level before the exam project is independent of the awareness and use of GitHub before the course. This sustains the importance of including tools in the context of an academic course.

The feedback on GitHub emphasizes that students appreciate the following functionalities: a common repository, a versioning system, a history of all modifications, an environment for synchronous development, support for conflicts resolution, and a sharing, teamwork, and coordination environment. As the most used and useful functionalities, students mentioned: pull/push, branching, GitHub Desktop, versioning, code merge, and conflict management. Among possible improvements for GitHub functionalities, students mentioned: a preview of the branch to be committed and its differences with the existent code, more efficient merge support, an enhancement of the GUI, the creation of activities (like Trello²³) and the automatic link to a branch with the same name (like Jira²⁴), a mobile app for history check, insert comments on commits for specific team members and notification of commits to all the team members. In 2020/2021, there emerged also proposals for improved communication support for team members (e.g., chats, calls).

²²They learned UML in a course in the second year on Object-Oriented Analysis and Design through UML and Design Pattern based on the corresponding Craig Larman's book.

²³Trello is a project management tool. <https://trello.com/>

²⁴Jira is a platform for projects' issue tracking. <https://www.atlassian.com/software/jira>

To investigate the teamwork, collaboration, and the interest of the team members in the others' work during the exam project, we inserted four specific questions. The first asks if a team member reads the code and documentation written by the other members. While the percentage of students not reading the others contribution is similar in time, we observe a slight growth from 15.6% in 2018/2019 to 18.2% in 2020/2021. The second asks if a team member thinks that the other team members read her/his code and documentation. The trend related to the first question is maintained also in this question: 62.2% in 2018/2019 vs 57.6% in 2020/2021 think that the other read their contribution. The third asks if students have any concerns about the others reading their code and documentation. Here the difference is more significant: 75.6% do not have any concern in 2018/2019 vs 100% in 2020/2021. The fourth concerns the use of notifications in GitHub: we notice a slight growth of their use from 8.9% in 2018/2019 to 12.1% in 2020/2021.

2) SONARQUBE

The feedback on SonarQube asks for its usefulness in the software quality analysis, its rating to evaluate the quality of a project, its use in the identification and removal of quality issues, its difficulties concerning the removal of issues, and missing functionalities. Currently, the SonarQube questionnaire counts 14 questions.

The feedback indicates that students had difficulties in using SonarQube mostly due to the different Java versions, use of Java with JavaScript or other programming languages, e.g., C#, and use of IDEs, e.g., Visual Studio. Based on the feedback, we managed to address these difficulties during the 2020/2021 course edition (by anticipating some of these potential issues and by presenting the tool in more detail). We reduced the percentage of students having difficulties in using SonarQube from 34.1% to 24.2. This trend is reflected also in the difficulties encountered by students to remove the quality issues. We reduced the percentage of students having difficulties in removing the quality issues from 20.5% in 2018/2019 to 12.1% in 2020/2021. The difficulties in removing quality issues derived from complex code based on nested *ifs*, unclear guidelines for issue removal, the need for code refactoring to overcome quality issues, the presence of security and vulnerability issues, and the presence of SQL access code. Furthermore, we also improved the students' perception on the usefulness of SonarQube in the software quality analysis: from 61.4% in 2018/2019 to 75.8% in 2020/2021. Students mentioned that SonarQube helped them in refactoring the code, removing bugs and smells, identifying issues hardly identifiable without SonarQube support, analyze in-depth every aspect of their code to have a good level of code quality. Students found the letter-based rating of SonarQube useful for the evaluation of their software quality: 72.7% in 2018/2019 and 81.8% in 2020/2021.

Students indicate as main drawbacks of SonarQube the lack of a guide for coverage analysis, the focus of the

tool on some minor issues, too many false positive issues, poor guidelines for quality improvement, and the need for an older version of Java installation. For SonarQube improvement, students mention the automatic resolution of the identified issues, the inclusion of the metrics provided by the Understand tool, the identification of architectural level issues (e.g., architectural smells), personalizing the quality issues to be identified, analysis of various branches in a GitHub project and the history analysis of quality issues for each project.

3) MICROSOFT PROJECT

The feedback on Microsoft Project asks for general information (e.g., awareness of its existence, eventual previous use), its usefulness for the exam project, its support for collaboration and task management, its role in the estimation of project development effort. Currently, the Microsoft Project questionnaire counts 18 questions, as for the 2018-2019 academic year. The awareness concerning Microsoft Project is constant in time: half of the students answered positively. There is a slight increase in the number of students using this tool or another tool for Gantt diagrams (e.g., Gantt project, EasyProject, Gantt in Excel): from 71.4% in 2018/2019 to 75% in 2020/2021. Students mentioned that they know about project management and Gantt diagrams from high school, their mates, the Internet, and work experiences.

We notice a slight decrease in the number of students capturing the benefits of a project management tool during the development of their exam project. For example, 88.1% considered it helpful in organizing the project work in 2018/2019 vs 85.3% in 2020/2021. 61.9% considered it helpful in the collaboration among team members in 2018/2019 vs 52.9% in 2020/2021. 76.2% considered it helpful in the management of the tasks distribution in the team in 2018/2019 vs 67.6%. These decreases may be due to several causes. Students mentioned that the amount of effort required for the project management was higher than its benefits because of the limited number of team members. We have anticipated this aspect during the seminars and stressed that project management is a key aspect in the success of a project and we provided also examples of projects that failed due to their poor management. Students mentioned that the main advantages of project management are a common way to define and control the tasks and their evolution in time, a high-level view of the project and in the same time its division into sub-tasks, the specification of priorities, the possibility to parallelize tasks, the uniform distribution of the tasks among the team members, and the improvement of their skills concerning time management.

4) UNDERSTAND

Questions about Understand relate to the usage of the tool during the development of the exam project. 78% of the students found Understand useful to identify structural antipatterns and dependency problems: when asked why, they

reported particularly appreciating the graph representation of the software dependencies and the metric treemap views, which allow comparing pairs of software metrics of the analyzed source code files. The remaining 22% did not find it useful mainly because they had difficulties in using it and defined it as “confusing”. In any case, about half of the students found examples of antipatterns in their projects. The reported instances comprise Local Butterflies, Global Breakable, and External Butterfly. The other half who did not find examples include also the students who, due to the above-mentioned difficulties, did not even try to look for them. The students who found antipatterns were also questioned about whether they tried to remove them and, in case of a positive answer, if the removal task was difficult. Only 52% tried to remove them and of this share, 75% of them reported that it was not difficult to carry out. Some students who, on the other hand, found it a difficult task, commented that the reason lies in the large amount of time required by the refactoring activity, which reduced the already limited available time given for the project development. Among the students who tried to remove the antipatterns, 70% found in Understand a valid support.

5) TRAVIS

Questions about Travis relate both to the tool and to the concept of Continuous Integration. 65% of the students did not know the concept of Continuous Integration and all but one had never heard about Travis. When asked whether the tool improves their productivity, the students divided into two sets: the ones reporting that they did not use it for their project and thus could not express a clear opinion about it, and the ones who indicated that helps them in saving time and keeping high the quality of their code. In any case, 66% of students answered “Medium” and 29% answered “High” when asked how much Travis is of support for smart-working, meaning that also the students who did not use it gave positive feedback about this aspect. Indeed, 82% of the students indicated that they are willing to use Travis in the future, expecting that they will use it (or another similar tool) in their future workplace.

C. RQ2: FEEDBACK ON THE CONTENTS OF THE COURSE FROM COMPANIES

During each edition of the course, we invite two companies to provide seminars on software development and its related issues. We aim to obtain indirect/implicit feedback on the contents of the course from the companies. A constant interaction with companies enables us to identify the actual needs of the companies and hence, identify further potential topics for our course.

Through these seminars, students capture that many contents and tools learned during the course are effectively used in the daily work in the companies. At the same time companies realized that, through this course, students are able to gain knowledge on topics relevant to the work done in the companies. This has been confirmed also through an

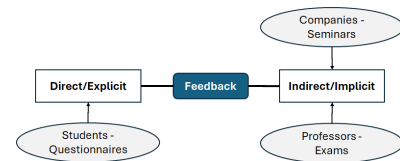


FIGURE 3. Towards a taxonomy for feedback on course content.

increasing demand in having students for an internship/stage in the companies, while during the past years, they were particularly interested in hiring students after their bachelor or master degree, having a lower interest in following students for an internship/stage required to obtain the degree (which can be either a stage at university or an external stage (internship) in a company). The contents that they outlined as more interesting for them are: 1) learning to use SonarQube and other tools such as Maven and Github and 2) the knowledge on design patterns, architectural patterns, and the SOLID design principles.

In particular, during a recent course edition, a first company provided a seminar entitled “*Business Production System (BPS): how the role of Best Practices influence the success of a software project.*” A senior developer introduced a case study, consisting of the migration of a Java project based on Spring and Hibernate towards microservices. He highlighted the importance of knowing design patterns and best practices, to enable fast evolution and scalability of the solution. He mentioned seven design patterns (that we teach in our course) and the SOLID principles. He also showed how they exploit SonarQube in their deploy pipeline to block or accept a new code commit.

A second company provided a seminar entitled “From Monolithic to Microservices - The challenges to face when migrating from a monolith to microservices”. During this seminar, a business manager introduced the students to the best practices and approaches to migrate a Java monolithic architecture towards a microservices solution. He reported a real case study he experienced which comprised the effective usage of 1) GitHub branches to maintain two different versions of the codebase (pre-migration and after-migration) and 2) the adoption of Maven to manage the build, package, test and deploy on the Docker platform.

After these seminars and after the exam, several students are interested in an internship in these companies or a hybrid thesis (a collaboration between university and company) on course topics.

D. TOWARDS A TAXONOMY FOR FEEDBACK ON TEACHING

In this section, we have described various sources of feedback for our course and its teaching. These sources can be categorized as direct/explicit from the main clients, i.e., the students, and indirect/implicit from the evaluation of students and from the interaction with companies. Figure 3 summarizes these sources of feedback and proposes a first taxonomy.

V. CONCLUSION AND FUTURE DEVELOPMENTS

This section summarizes the answers to the RQ, discusses our findings, and identifies future development directions.

A. ANSWERS TO RQ

We now answer the sub-research questions of our main research question as follows:

RQ: Which are the main challenges faced in a continuously evolving software engineering course?:

1) RQ1: WHICH IS THE MAIN FEEDBACK ON THE COURSE THROUGH THE QUESTIONNAIRES AND EXAMS PROJECTS?

RQ1: Finding 1: First, students understand and apply design principles and patterns in their exam project, and become aware of how these concepts greatly help them in improving the quality of their software.

RQ1: Finding 2: Second, students are enthusiastic to learn about tools and use them in software development and software quality assessment. They are more and more aware of the existence of various tools, as well as eager to discover new ones.

RQ1: Finding 3: Third, projects help students apply the contents of the course to a practical example. Projects also help professors capture the topics not properly understood by the students and identify contents that need improvement in the teaching phase or new content to be considered.

RQ1 Finding 4: Fourth, students become aware of the various technical (e.g., design principles, design and architectural patterns, software quality assessment) and soft skills (e.g., teamwork, collaboration, management tasks) needed for software project development.

2) RQ2: HOW DID THE COURSE CONTENTS EVOLVE DURING THE LAST DECADE?

RQ2: Finding 1: First, we continuously revised, refined, and improved the course contents based on the various types of feedback. This ensures efficient and effective teaching.

RQ2: Finding 2: Second, we gradually introduced DevOps related concepts. We introduced new tools for software development (e.g., GitHub, Travis) and software analysis (Understand). We added design principles and best practices in software engineering.

RQ2: Finding 3: We focused both on hard and soft skills required in software engineering. It is equally important that students are aware of the technical knowledge needed to develop software and of the social abilities (collaboration, communication, teamwork, management) to make the development of software a success.

B. GUIDELINES FOR CONTINUOUS EVOLUTION ON TEACHING SE

Our main guidelines for applying continuous evolution in software engineering teaching (and potentially in any other field) are:

Guideline 1: Continuously revise, refine, and improve the teaching of the current content of a course based on various types of feedback.

The same SE content may be taught in different ways. The student generations also evolve over time: they have different backgrounds and experiences. The industry needs are dynamic. Using various types of feedback, professors may have a more precise perspective on how the course content may be presented to reach its scope. In this context, this guideline aims to ensure that the content is delivered in an appropriate way for the current students.

Guideline 2: Continuously change the course content based on the evolution of the field and society.

SE is evolving continuously. Industry expectations and student expectations are also changing. Students background is evolving over time. Course content should reflect the innovation in the field, hence new content should be added. This has the side effect of revising the current content to make room for the new topics. Some course contents may become obsolete and should be removed. Or, they may be shrunk. Hence, changes may imply removal, revision, and addition. In this context, this guideline indicates that new content should be periodically included in a course to achieve its objectives, and at the same time, current content should be revised (removed or condensed).

These guidelines may lead to an increased workload for the professors, which may become unsustainable in some cases. To overcome this, we suggest either performing limited changes every year, or performing changes periodically (every 3-4 years, instead of every year).

C. LIMITATIONS OF OUR RESEARCH

The main limitations of our research may be generated by the consideration of one course, the involvement only of the professors of this course, and the number of students in the course. However, we considered several editions of our SE course during a decade. All the professors involved in the course during these years have aligned opinions on the results. These ensure a certain confidence in the described results. The population of students involved during these years is around 400+, which may represent a limited number considering the first year mandatory courses with 400+ students per edition. However, for an optional course in the third year, at our university, the student population is representative.

D. FURTHER DEVELOPMENTS

In the future, we plan to continue gathering feedback of various types and from various sources, and to improve our SE course. For example, we plan to invite our previous students who work in industry or in universities to share their experiences related to SE activities. Moreover, we plan to extend our lessons about DevOps. We started focusing on CI and tools to enable it, however, we aim to introduce the practice of Infrastructure as a Code (IaaS) [51], the concept and tools for Continuous Delivery and eventually to

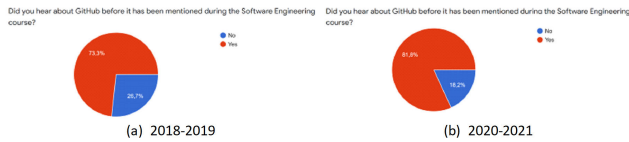


FIGURE 4. Awareness concerning the existence of GitHub.



FIGURE 5. Use of GitHub before the Software engineering course.

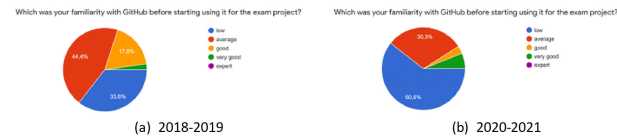


FIGURE 6. Familiarity with GitHub before the exam project.



FIGURE 7. Familiarity with GitHub after the exam project.



FIGURE 8. Usefulness of GitHub during the exam project.

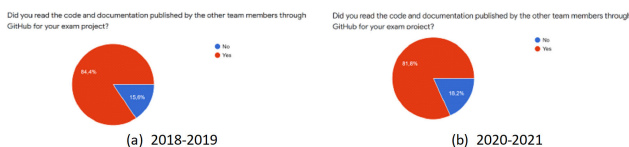


FIGURE 9. Reading code/documentation of the other team members.



FIGURE 10. Reading code/documentation by the other team members.



FIGURE 11. Concerns about the reading code/documentation by the other team members.



FIGURE 12. Use of GitHub notifications during the exam project.



FIGURE 13. Difficulties in using SonarQube.



FIGURE 14. Difficulties in removing quality issues with SonarQube.

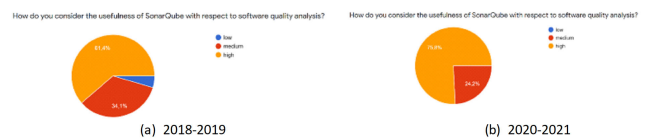


FIGURE 15. Usefulness of SonarQube for software quality analysis.

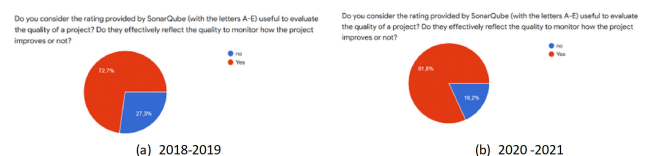


FIGURE 16. Usefulness of SonarQube rating for software quality analysis.

enable students to experiment with cloud platforms such as Microsoft Azure²⁵ and AWS.²⁶ Our idea is to progressively cover all the aspects of the development life-cycle and exploit the occasion given by the exam project to make students produce a complete solution, in terms of design, development and deployment.

Another direction in the future will consider topics like prompt engineering, Large Language Models (LLM), and ChatGPT in SE education [52], [53], [54], [55]. We would

²⁵<https://azure.microsoft.com/>

²⁶<https://aws.amazon.com/>

like to introduce students to the advantages, limitations, and issues of AI-based approaches/tools in SE since the use of AI in SE is growing every day [56].

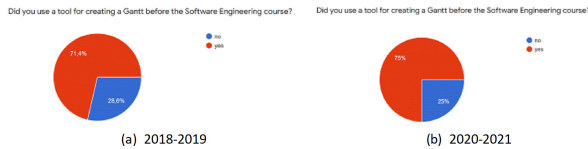


FIGURE 17. Use of a tool for creating Gantt diagrams before the Software Engineering course.



FIGURE 18. Usefulness of a Gantt diagram for the organization of the project work.



FIGURE 19. Usefulness of a Gantt diagram for the collaboration among team members.

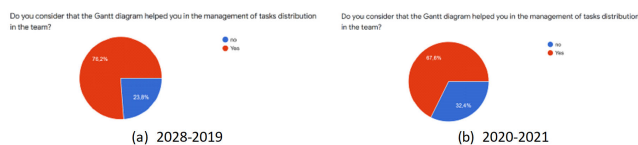


FIGURE 20. Usefulness of a Gantt diagram for the task management in the team.

VI. FUNDING DECLARATION

No funding has been received for this research.

APPENDIX STUDENTS' FEEDBACK EXAMPLES

The figures showing some of the student feedback during the 2018-2019 and 2020-2021 academic years are presented below.

REFERENCES

- [1] T. Ahtee and T. Poranen, "Risks in students' software projects," in *Proc. 22nd Conf. Softw. Eng. Educ. Training*, Feb. 2009, pp. 154–157, doi: [10.1109/CSEET.2009.31](https://doi.org/10.1109/CSEET.2009.31).
- [2] D. Broman, K. Sandahl, and M. Abu Baker, "The company approach to software engineering project courses," *IEEE Trans. Educ.*, vol. 55, no. 4, pp. 445–452, Nov. 2012, doi: [10.1109/TE.2012.2187208](https://doi.org/10.1109/TE.2012.2187208).
- [3] M. Ardis, D. Budgen, G. W. Hislop, J. Offutt, M. Sebern, and W. Visser, "SE 2014: Curriculum guidelines for undergraduate degree programs in software engineering," *Computer*, vol. 48, no. 11, pp. 106–109, Nov. 2015.
- [4] M. Hofbauer, C. Bachhuber, C. Kuhn, and E. Steinbach, "Teaching software engineering as programming over time," in *Proc. IEEE/ACM 4th Int. Workshop Softw. Eng. Educ. Next Gener. (SEENG)*, May 2022, pp. 51–58, doi: [10.1145/3528231.3528353](https://doi.org/10.1145/3528231.3528353).
- [5] C. Gupta, "The impact and measurement of today's learning technologies in teaching software engineering course using design-based learning and project-based learning," *IEEE Trans. Educ.*, vol. 65, no. 4, pp. 703–712, Nov. 2022.
- [6] F. Arcelli Fontana and C. Raibulet, "Students' feedback in using GitHub in a project development for a software engineering course," in *Proc. ACM Conf. Innov. Technol. Comput. Sci. Educ.*, Bologna, Italy, Jun. 2017, p. 380, doi: [10.1145/3059009.3072984](https://doi.org/10.1145/3059009.3072984).
- [7] C. Raibulet and F. Arcelli Fontana, "Collaborative and teamwork software development in an undergraduate software engineering course," *J. Syst. Softw.*, vol. 144, pp. 409–422, Oct. 2018, doi: [10.1016/j.jss.2018.07.010](https://doi.org/10.1016/j.jss.2018.07.010).
- [8] C. Raibulet, F. A. Fontana, and I. Pigazzini, "Teaching software engineering tools to undergraduate students," in *Proc. 11th Int. Conf. Educ. Technol. Comput.*, Oct. 2019, pp. 262–267, doi: [10.1145/3369255.3369300](https://doi.org/10.1145/3369255.3369300).
- [9] C. Raibulet, F. A. Fontana, and I. Pigazzini, "Hints on designing and running project-based exams for a software engineering course," in *Proc. IEEE/ACM 1st Int. Workshop Designing Running Project-Based Courses Softw. Eng. Educ. (DREE)*, May 2022, pp. 15–19, doi: [10.1145/3524487.3527355](https://doi.org/10.1145/3524487.3527355).
- [10] R. Hobeck, I. Weber, L. Bass, and H. Yasar, "Teaching DevOps: A tale of two universities," in *Proc. ACM SIGPLAN Int. Symp. SPLASH-E*, Oct. 2021, pp. 26–31, doi: [10.1145/3484272.3484962](https://doi.org/10.1145/3484272.3484962).
- [11] M. Fernandes, S. Ferino, A. Fernandes, U. Kulesza, E. Aranha, and C. Treude, "DevOps education: An interview study of challenges and recommendations," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, May 2022, pp. 90–101, doi: [10.1145/3510456.3514152](https://doi.org/10.1145/3510456.3514152).
- [12] O. E. Holo, E. N. Kveim, M. S. Sysne, L. H. Taraldsen, and F. O. Haara, "A review of research on teaching of computer programming in primary school mathematics: Moving towards sustainable classroom action," *Educ. Inquiry*, vol. 14, no. 4, pp. 513–528, May 2022, doi: [10.1080/20004508.2022.2072575](https://doi.org/10.1080/20004508.2022.2072575).
- [13] C. Troussas, A. Krouska, and C. Sgouropoulou, "Improving learner-computer interaction through intelligent learning material delivery using instructional design modeling," *Entropy*, vol. 23, no. 6, p. 668, May 2021, doi: [10.3390/e23060668](https://doi.org/10.3390/e23060668).
- [14] H. Belmar, "Review on the teaching of programming and computational thinking in the world," *Frontiers Comput. Sci.*, vol. 4, Oct. 2022, Art. no. 997222. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fcomp.2022.997222>
- [15] K. Pažur Anicic and Z. Stacic, "Teaching methods in software engineering: A systematic review," *IEEE Softw.*, vol. 39, no. 6, pp. 73–79, Nov. 2022.
- [16] J. Feliciano, M.-A. Storey, and A. Zagalsky, "Student experiences using GitHub in software engineering courses: A case study," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. Companion (ICSE-C)*, May 2016, pp. 422–431.
- [17] A. Zagalsky, J. Feliciano, M.-A. Storey, Y. Zhao, and W. Wang, "The emergence of GitHub as a collaborative platform for education," in *Proc. 18th ACM Conf. Comput. Supported Cooperat. Work Social Comput.*, Vancouver, BC, Canada, Feb. 2015, pp. 1906–1917.
- [18] R. Roveda, F. Arcelli Fontana, C. Raibulet, M. Zanolini, and F. Rampazzo, "Does the migration to GitHub relate to internal software quality?" in *Proc. 12th Int. Conf. Eval. Novel Approaches Softw. Eng.*, Porto, Portugal, 2017, pp. 293–300, doi: [10.5220/0006367402930300](https://doi.org/10.5220/0006367402930300).
- [19] R. Francese, C. Gravino, M. Risi, G. Scanniello, and G. Tortora, "Using project-based-learning in a mobile application development course—An experience report," *J. Vis. Lang. Comput.*, vol. 31, pp. 196–205, Dec. 2015, doi: [10.1016/j.jvlc.2015.10.019](https://doi.org/10.1016/j.jvlc.2015.10.019).
- [20] R. Francese, C. Gravino, M. Risi, G. Scanniello, and G. Tortora, "On the experience of using git-hub in the context of an academic course for the development of apps for smart devices," in *Proc. Int. Conf. Distrib. Multimedia Syst.*, Sep. 2015, pp. 292–299, doi: [10.18293/dms2015-003](https://doi.org/10.18293/dms2015-003).
- [21] P. Patani, S. Tiwari, and S. S. Rathore, "The impact of GitHub on students' learning and engagement in a software engineering course," *Comput. Appl. Eng. Educ.*, vol. 32, no. 5, Sep. 2024, Art. no. e22775, doi: [10.1002/cae.22775](https://doi.org/10.1002/cae.22775).
- [22] Y. Bai, L. Chen, G. Yin, X. Mao, Y. Deng, T. Wang, Y. Lu, and H. Wang, "Quantitative analysis of learning data in a programming course," in *Proc. Int. Workshops Database Syst. Adv. Appl.*, Suzhou, China, Jan. 2017, pp. 436–441, doi: [10.1007/978-3-319-55705-2_37](https://doi.org/10.1007/978-3-319-55705-2_37).
- [23] P. Lauvås, "Analyzing Student code after introducing portfolio assessment and automated testing," in *Proc. 7th Int. Conf. Inf. Knowl. Technol.*, vol. 23, Nov. 2015, pp. 1–8. [Online]. Available: <https://ojs.bibsys.no/index.php/Nokobit/article/view/286>
- [24] D. Falessi and P. Kruchten, "Five reasons for including technical debt in the software engineering curriculum," in *Proc. Eur. Conf. Softw. Archit. Workshops*, Sep. 2015, pp. 1–4, doi: [10.1145/2797433.2797462](https://doi.org/10.1145/2797433.2797462).

- [25] D. Delgado-Gómez, A. Velasco, J. Aponte, and A. Marcus, "Evolving a project-based software engineering course: A case study," in *Proc. 30th IEEE Conf. Softw. Eng. Educ. Training*, Nov. 2017, pp. 77–86, doi: [10.1109/CSEET.2017.22](https://doi.org/10.1109/CSEET.2017.22).
- [26] P. H. D. Andrade Gomes, R. E. Garcia, G. Spadon, D. M. Eler, C. Olivete, and R. C. M. Correia, "Teaching software quality via source code inspection tool," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2017, pp. 1–8, doi: [10.1109/FIE.2017.8190658](https://doi.org/10.1109/FIE.2017.8190658).
- [27] G. Rong, S. Gu, H. Zhang, and D. Shao, "DevOpsEnvoy: An education support system for DevOps," in *Proc. IEEE 30th Conf. Softw. Eng. Educ. Training (CSEET)*, Savannah, GA, USA, Nov. 2017, pp. 37–46, doi: [10.1109/CSEET.2017.17](https://doi.org/10.1109/CSEET.2017.17).
- [28] D. P. Groth and M. P. Hottell, "Designing and developing an informatics capstone project course," in *Proc. 19th Conf. Softw. Eng. Educ. Training (CSEET)*, Apr. 2006, pp. 61–68, doi: [10.1109/CSEET.2006.15](https://doi.org/10.1109/CSEET.2006.15).
- [29] Z. Car, H. Belani, and K. Pripuzic, "Teaching project management in academic ICT environments," in *Proc. EUROCON-Int. Conf. 'Comput. Tool'*, Sep. 2007, pp. 2403–2409.
- [30] A.-J. Stoica and S. Islam, "Educational methods for software and systems development," in *Proc. 15th Int. Conf. Interact. Collaborative Learn. (ICL)*, Sep. 2012, pp. 1–9.
- [31] J. Garcia, D. Popescu, G. Edwards, and N. Medvidović, "Toward a catalogue of architectural bad smells," in *Proc. 5th Int. Conf. Quality Softw. Archit. Adapt. Softw. Syst.*, Jan. 2009, pp. 146–162, doi: [10.1007/978-3-642-02351-4_10](https://doi.org/10.1007/978-3-642-02351-4_10).
- [32] O. Cico, L. Jaccheri, A. Nguyen-Duc, and H. Zhang, "Exploring the intersection between software industry and software engineering education—A systematic mapping of software engineering trends," *J. Syst. Softw.*, vol. 172, Feb. 2021, Art. no. 110736. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121220301667>
- [33] J.-G. Schneider, P. W. Eklund, K. Lee, F. Chen, A. Cain, and M. Abdelrazek, "Adopting industry agile practices in large-scale capstone education," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, Oct. 2020, pp. 119–129.
- [34] K. Ryan, "We should teach our students what industry doesn't want," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, Oct. 2020, pp. 103–106.
- [35] A. Radermacher, G. Walia, and D. Knudson, "Investigating the skill gap between graduating students and industry expectations," in *Companion Proc. 36th Int. Conf. Softw. Eng.*, May 2014, pp. 291–300.
- [36] J. Vanhanen, T. O. A. Lehtinen, and C. Lassenius, "Software engineering problems and their relationship to perceived learning and customer satisfaction on a software capstone project," *J. Syst. Softw.*, vol. 137, pp. 50–66, Mar. 2018.
- [37] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proc. IEEE*, vol. 68, no. 9, pp. 1060–1076, Sep. 1980.
- [38] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*. Upper Saddle River, NJ, USA: Prentice-Hall, 2001.
- [39] E. Gamma, R. Helm, R. E. Johnson, and J. M. Vlissides, *Design Patterns: Abstraction and Reuse of Object-Oriented Design*. Berlin, Germany: Springer, 1994.
- [40] M. Fowler, *Patterns of Enterprise Application Architecture*. Reading, MA, USA: Addison-Wesley, 2012.
- [41] *A Guide to the Project Management Body of Knowledge (PMBOK Guide)—Seventh Edition and The Standard for Project Management*, Project Management Institute, Inc., Newtown Square, PA, USA, 2021. [Online]. Available: <https://www.pmi.org/standards/pmbok>
- [42] R. C. Martin, "Design principles and design patterns," *Object Mentor*, vol. 1, no. 34, p. 597, 2000.
- [43] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [44] G. Suryanarayana, G. Samarthyam, and T. Sharma, *Refactoring for Software Design Smells: Managing Technical Debt*, 1st ed., San Francisco, CA, USA: Morgan Kaufmann, 2014.
- [45] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol.*, Sep. 2014, pp. 91–100.
- [46] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Reading, MA, USA: Addison-Wesley, 2015.
- [47] I. Alves and C. Rocha, "Qualifying software engineers undergraduates in DevOps—Challenges of introducing technical and non-technical concepts in a project-oriented course," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, Madrid, Spain, May 2021, pp. 144–153, doi: [10.1109/ICSE-SEET52601.2021.00024](https://doi.org/10.1109/ICSE-SEET52601.2021.00024).
- [48] M. Fowler and M. Foemmel. (2006). *Continuous Integration*. ThoughtWorks. [Online]. Available: <http://www.thoughtworks.com/ContinuousIntegration.pdf>
- [49] M. Soni, "End to end automation on cloud with build pipeline: The case for DevOps in insurance industry, continuous integration, continuous testing, and continuous delivery," in *Proc. IEEE Int. Conf. Cloud Comput. Emerg. Markets (CCEM)*, Nov. 2015, pp. 85–89.
- [50] V. Rajlich, "Teaching developer skills in the first software engineering course," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, May 2013, pp. 1109–1116, doi: [10.1109/ICSE.2013.6606661](https://doi.org/10.1109/ICSE.2013.6606661).
- [51] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [52] V. D. Kirova, C. S. Ku, J. R. Laracy, and T. J. Marlowe, "Software engineering education must adapt and evolve for an LLM environment," in *Proc. 55th ACM Tech. Symp. Comput. Sci. Educ.*, Mar. 2024, pp. 666–672, doi: [10.1145/3626252.3630927](https://doi.org/10.1145/3626252.3630927).
- [53] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," 2023, *arXiv:2303.07839*.
- [54] S. Rasnayaka, G. Wang, R. Shariffdeen, and G. Neelakanta Iyer, "An empirical study on usage and perceptions of LLMs in a software engineering project," in *Proc. IEEE/ACM Int. Workshop Large Lang. Models Code (LLM4Code)*, Apr. 2024, pp. 111–118.
- [55] J. Pereira, J.-M. López, X. Garmendia, and M. Azanza, "Leveraging open source LLMs for software engineering education and training," in *Proc. 36th Int. Conf. Softw. Eng. Educ. Training (CSEET)*, Aug. 2024, pp. 1–10.
- [56] H. Sofian, N. A. M. Yunus, and R. Ahmad, "Systematic mapping: Artificial intelligence techniques in software engineering," *IEEE Access*, vol. 10, pp. 51021–51040, 2022.



CLAUDIA RAIBULET received the master's degree in computer science from Politehnica Bucharest, Romania, and the Ph.D. degree in computer science from the Politecnico di Torino, Italy.

She is currently an Associate Professor with the University of Milano-Bicocca, Italy. Previously, she was an Assistant Professor at the University of Milano-Bicocca, Vrije Universiteit Amsterdam, The Netherlands, and Politehnica Bucharest. She is the co-author of more than 100 papers published in international journals, conferences, and workshops. Her research interests include software engineering, including self-adaptive systems, software architectures, software quality assessment, object-oriented modeling, model-driven approaches, reverse engineering, and runtime evaluation.

Prof. Raibulet is a member of the editorial board of *Information and Software Technology* journal and an Associate Editor of *IEEE Access* journal. She is involved in referee activities for various international journals, as well as in organizing and program committees for international conferences and workshops.



FRANCESCA ARCELLI FONTANA is currently a Full Professor with the University of Milano-Bicocca, Italy, where she is also the Head of the Software Evolution and Reverse Engineering Laboratory. Her research interests include software engineering, in particular software evolution and reverse engineering, architectural smell detection, program comprehension, software quality assessment, and managing technical debt.

...